FAIRLANE · PHASE II · SHARED STORAGE TRACK

NFS Shared Storage in Fairlane

How we set it up, why it is built this way, and how a new user is registered

Team Orchestrators · Fairlane: A Fair and Fault-Tolerant Scheduler for Distributed ML Training Workloads

1 The problem NFS solves
2 Architecture
3 Folder layout & permissions
4 Identity: why the numbers matter
5 Security model
6 Our helper scripts
7 Setting it up
8 Demo: registering a new user
9 Summary flowchart · 10 Q&A

1 The problem NFS solves

In Fairlane, a user submits a training job by giving the scheduler a **path** to a script, not the file itself. The scheduler then sends that job to whichever worker machine has a free GPU. That raises two questions:

- **How does the worker open a file that lives on someone else's machine?**
- **Where does the job's log go, so the user (and, once it is built, the dashboard) can read it?**

We could have written our own upload/download code, but that is extra code, extra bugs and extra latency. Instead we use **NFS (Network File System)**. NFS makes one folder on a server look like an ordinary local folder on every other machine. So:

> **Users** put scripts and data in their own folder on the share.

> **Workers** read the script straight from the share and write the log back to it.

> **The dashboard** (planned) will read logs from the same share. No copying anywhere.

This is the same pattern used by Slurm-style GPU clusters (for example NVIDIA DeepOps): shared storage plus a scheduler that only passes paths around.

2 Architecture

One machine is the **NFS server**: it owns the real folder `/srv/fairlane` and *exports* it. Every other machine is a **client**: it *mounts* that folder at `/mnt/shared_jobs`. Reading or writing `/mnt/shared_jobs` on a client really reads or writes `/srv/fairlane` on the server.

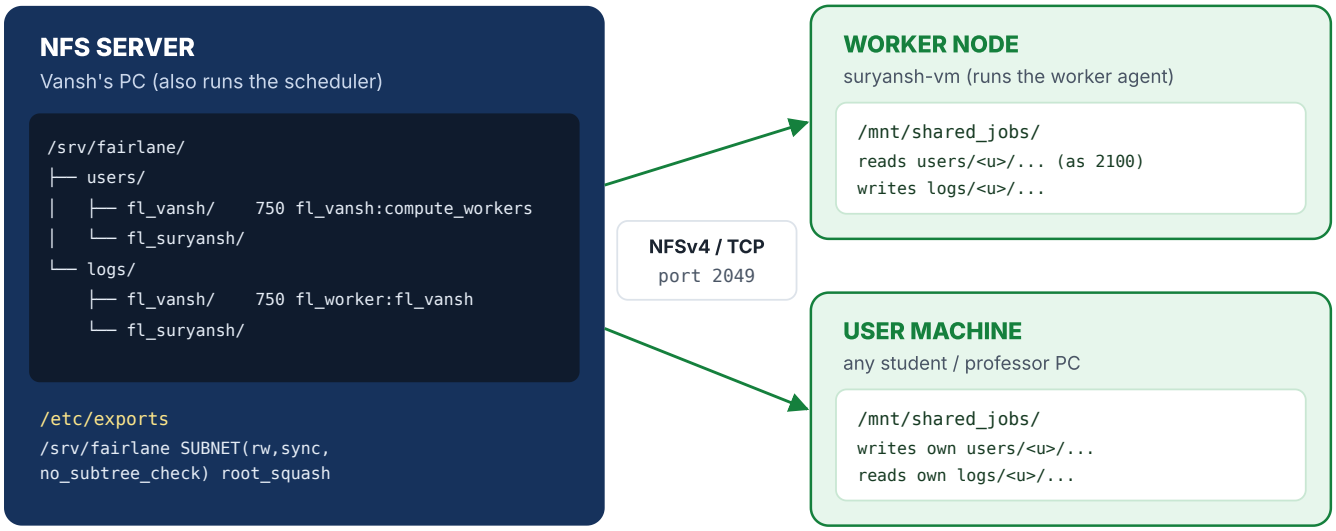


Figure 1. One server, many clients. In our demo the server is Vansh's PC and the client is suryansh-vm.

DECISION	WHAT WE CHOSE	WHY
Server folder	`/srv/fairlane`	`/srv` is the standard Linux place for data a machine serves to others. Keeping it separate from the mount point matters when the server is also a client.
Client mount point	`/mnt/shared_jobs` on **every** machine	The same path everywhere, so a path means the same thing on every machine.
Paths in the database	Relative, e.g. `users/fl_vansh/job_7/train.py`	The scheduler never depends on where the share is mounted. The worker adds its own root in front.
Protocol	NFSv4 over TCP, port 2049	One port to open in the firewall; v4 is the modern default on Ubuntu 24.04.

3 Folder layout & permissions

The share has two areas. **users/** is where people put their work; **logs/** is where workers write results. We deliberately separated them so a worker never needs write access to a user's code.

```
/srv/fairlane/
├── users/
│   ├── fl_vansh/      fl_vansh : compute_workers  750
│   │   └── job_7/train.py
│   └── fl_suryansh/
└── logs/
    ├── fl_vansh/      fl_worker : fl_vansh  750
    └── fl_suryansh/
```

How to read "750": three digits for *owner*, *group*, *everyone else*. 7 = read + write + enter, 5 = read + enter, 0 = nothing.

So in `users/fl_vansh` the owner can do everything, the worker group can only read, and other students get nothing. In `logs/fl_vansh` it's flipped: the worker writes, the user can read.

	users/fl_vansh (code)	logs/fl_vansh (results)
fl_vansh (the owner)	✓ read + write	✓ read
fl_worker (runs jobs)	✓ read only	✓ read + write
fl_suryansh (another user)	× no access	× no access

In our NFS tests we also checked the container case: a job started with `--user 2100:2100` (fl_worker) and the user's folder mounted read-only can read the script and write its log, but cannot change anyone's code. (The worker agent in the live demo runs plain processes; switching it to containers is planned.)

4 Identity: why the numbers matter

This is the part most people get wrong with NFS. **NFS does not send usernames; it sends numbers (UID and GID).** If `fl_vansh` is user 2001 on the server but 1005 on a client, then on that client his files appear to belong to somebody else, and the permissions above silently stop working.

So we made a single source of truth, `nfs/registry.conf`, where every number is assigned once and used on every machine:

```
WORKER_USER=fl_worker
WORKER_GROUP=compute_workers
WORKER_ID=2100

# User IDs: 2001-2099. Next free: 2003.
USERS="fl_vansh:2001 fl_suryansh:2002"
```

`fl_worker / compute_workers`	**2100**
`fl_vansh`	**2001**
`fl_suryansh`	**2002**
`next free`	**2003**

Rules: numbers are only assigned in this file, never reused, and each user also gets a private group with the same number.

5 Security model

SETTING	WHAT IT DOES
`rw,sync,no_subtree_check`	Read-write share. `sync` writes to disk before confirming, which is safe if the server crashes. `no_subtree_check` is recommended by the exports manual.
Export only to our subnet	`/srv/fairlane 10.190.94.0/24(...)` - machines outside the lab network cannot mount it.
`root_squash` **(default)**	A client's `root` is treated as `nobody` on the share, so having `sudo` on your laptop does **not** give you root over everyone's files.
750 folders + separate logs area	Users are isolated from each other; workers can read code but never modify it.

KNOWN LIMITATION Plain NFS trusts the UID a client claims. Someone with root on their own machine could create a local user with another person's number. Fixing this needs Kerberos (NFSv4 + krb5), which is out of scope, so we assume a trusted lab network. We state this openly in the report.

6 Our helper scripts

Doing all of this by hand on every machine is error-prone, so we wrote five small scripts in the `nfs/` folder. All of them are safe to re-run.

SCRIPT	WHERE	WHAT IT DOES
`registry.conf`	-	The list of fixed IDs (not a script, the data the scripts read).
`sync_accounts.sh`	every machine	Creates the worker account and every user with the exact numbers from the registry. Leaves correct accounts alone and **stops with an error** if a name or number is already used differently.
`server_dirs.sh`	server	Runs sync_accounts.sh, then creates or repairs `users/<u>` and `logs/<u>` with the right owner and mode 750.
`join.sh`	each client	Installs the NFS client, runs sync_accounts.sh, mounts the share at `/mnt/shared_jobs` (optionally permanently via `/etc/fstab`), then runs verify.sh.
`verify.sh`	any machine	Five checks: share mounted; worker can read `users/`; worker can write `logs/`; worker **cannot** write `users/`; one user **cannot** read another's folder.

7 Setting it up

On the server (Vansh's PC), once

1. **Start the NFS server.**

   RUN Vansh's PC (server)

   ```
   sudo systemctl enable --now nfs-server
   ```

2. **Allow our network.** In `/etc/exports`, one line for the server itself and one for today's subnet (a hotspot gives new IPs each time):

   ```
   /srv/fairlane 127.0.0.1(rw,sync,no_subtree_check) 10.190.94.0/24(rw,sync,no_subtree_check)
   ```

   then apply it:

   RUN Vansh's PC (server)

   ```
   sudo exportfs -ra
   sudo exportfs -v
   ```

3. **Create accounts and folders, and self-test.**

   RUN Vansh's PC (server)

   ```
   sudo bash nfs/server_dirs.sh
   sudo bash nfs/join.sh 127.0.0.1
   ```

On each client (e.g. suryansh-vm)

BEFORE YOU START *the client needs its own copy of the `nfs/` folder (all five files). Push it to `main` once, then on the client run `git fetch origin main && git checkout origin/main -- nfs`. The client's IP must also be inside the subnet written in `/etc/exports`.*

1. **Join with one command.** It installs the client, creates the same accounts, mounts and verifies.

RUN suryansh-vm (client)

```
sudo bash nfs/join.sh <SERVER_IP>
```

EXPECTED OUTPUT

```
PASS  share mounted at /mnt/shared_jobs
PASS  worker can read users/fl_vansh
PASS  worker can write logs/fl_vansh
PASS  worker cannot write users/fl_vansh
PASS  fl_suryansh cannot read fl_vansh's folder
ALL CHECKS PASSED
```

AFTER A REBOOT *the mount is gone unless it was made permanent. Either re-run `join.sh`, or join once with `sudo bash nfs/join.sh <SERVER_IP> --persist`, which adds an `/etc/fstab` entry (only worth it on a network whose IPs don't change).*

8 Demo: registering a new user

This is what we show the evaluator. We register a new team member, **fl_harsh**, and give him the next free number, **2003**. The point to make: *adding a user is one line in a file and one command per machine, and because the number is fixed, his files belong to him on every machine.*

1 Edit registry → 2 Server: create → 3 Client: sync → 4 Prove it

1 Assign the number in the registry — Vansh's PC

Open `nfs/registry.conf` and add the user. We also bump "Next free" so nobody reuses 2003.

```
# User IDs: 2001-2099. Next free: 2004. Never reuse a number.
USERS="fl_vansh:2001 fl_suryansh:2002 fl_harsh:2003"
```

Quick way without an editor:

RUN Vansh's PC (server)

```
sed -i 's/Next free: 2003/Next free: 2004/; s/fl_suryansh:2002"/fl_suryansh:2002
fl_harsh:2003"/' nfs/registry.conf
```

What's happening: nothing on the system changes yet. We are only deciding the identity, in one place, before any machine uses it.

2 Create the account and folders on the server

Vansh's PC

RUN Vansh's PC (server)

```
sudo bash nfs/server_dirs.sh
```

EXPECTED OUTPUT

```
created group fl_harsh (2003)
created user fl_harsh (2003)
OK: accounts on vansh-Inspiron-7577 match the registry
folders ready for fl_vansh
folders ready for fl_suryansh
folders ready for fl_harsh
(then an ls -ln listing of users/ and logs/, now including 2003)
```

What's happening: the script creates user and group 2003, then makes `users/fl_harsh` (owner fl_harsh, group compute_workers, 750) and `logs/fl_harsh` (owner fl_worker, group fl_harsh, 750). Existing users are left untouched; that's why it's safe to re-run.

3 Create the same account on every client

suryansh-vm

Make the same registry edit on the client (in real use: Vansh commits the file and everyone runs `git pull`), then:

RUN suryansh-vm (client)

```
sudo bash nfs/sync_accounts.sh
```

EXPECTED OUTPUT

```
created group fl_harsh (2003)
created user fl_harsh (2003)
OK: accounts on Ubuntu match the registry
```

What's happening: the client doesn't need to create folders, because it already *sees* the server's folders through the mount. It only needs to know that number 2003 means fl_harsh, so file ownership and permissions make sense here too.

4 Prove it works

suryansh-vm

RUN suryansh-vm (client)

```
id fl_harsh
ls -ln /mnt/shared_jobs/users
sudo -u fl_harsh ls /mnt/shared_jobs/users/fl_vansh
```

EXPECTED OUTPUT

```
uid=2003(fl_harsh) gid=2003(fl_harsh) groups=2003(fl_harsh)
drwxr-x--- 2 2003 2100 ... fl_harsh
drwxr-x--- 2 2002 2100 ... fl_suryansh
drwxr-x--- 2 2001 2100 ... fl_vansh
ls: cannot open directory '/mnt/shared_jobs/users/fl_vansh': Permission denied
```

How to explain each line:

- `id` shows the account exists on the client with exactly the number from the registry.
- `ls -ln` shows *numbers* instead of names. The new folder, created on the server, is owned by 2003 here too. That's the matching-UID rule working.
- *Permission denied* shows isolation: a new user cannot look into another user's folder.

5 Optional: the new user's work reaches the worker

On the server, fl_harsh saves a script in his folder. On the client, the worker account reads it, and fl_suryansh still can't.

RUN Vansh's PC (server)

```
echo 'print("hello from harsh")' | sudo -u fl_harsh tee /srv/fairlane/users/fl_harsh/train.py
```

RUN suryansh-vm (client)

```
sudo -u fl_worker cat /mnt/shared_jobs/users/fl_harsh/train.py
sudo -u fl_suryansh cat /mnt/shared_jobs/users/fl_harsh/train.py
```

EXPECTED OUTPUT

```
print("hello from harsh")
cat: /mnt/shared_jobs/users/fl_harsh/train.py: Permission denied
```

Why this matters: it is exactly what happens when fl_harsh submits a job: he writes the script, the scheduler passes the path `users/fl_harsh/train.py`, and the worker reads it as fl_worker. Other students still can't see it.

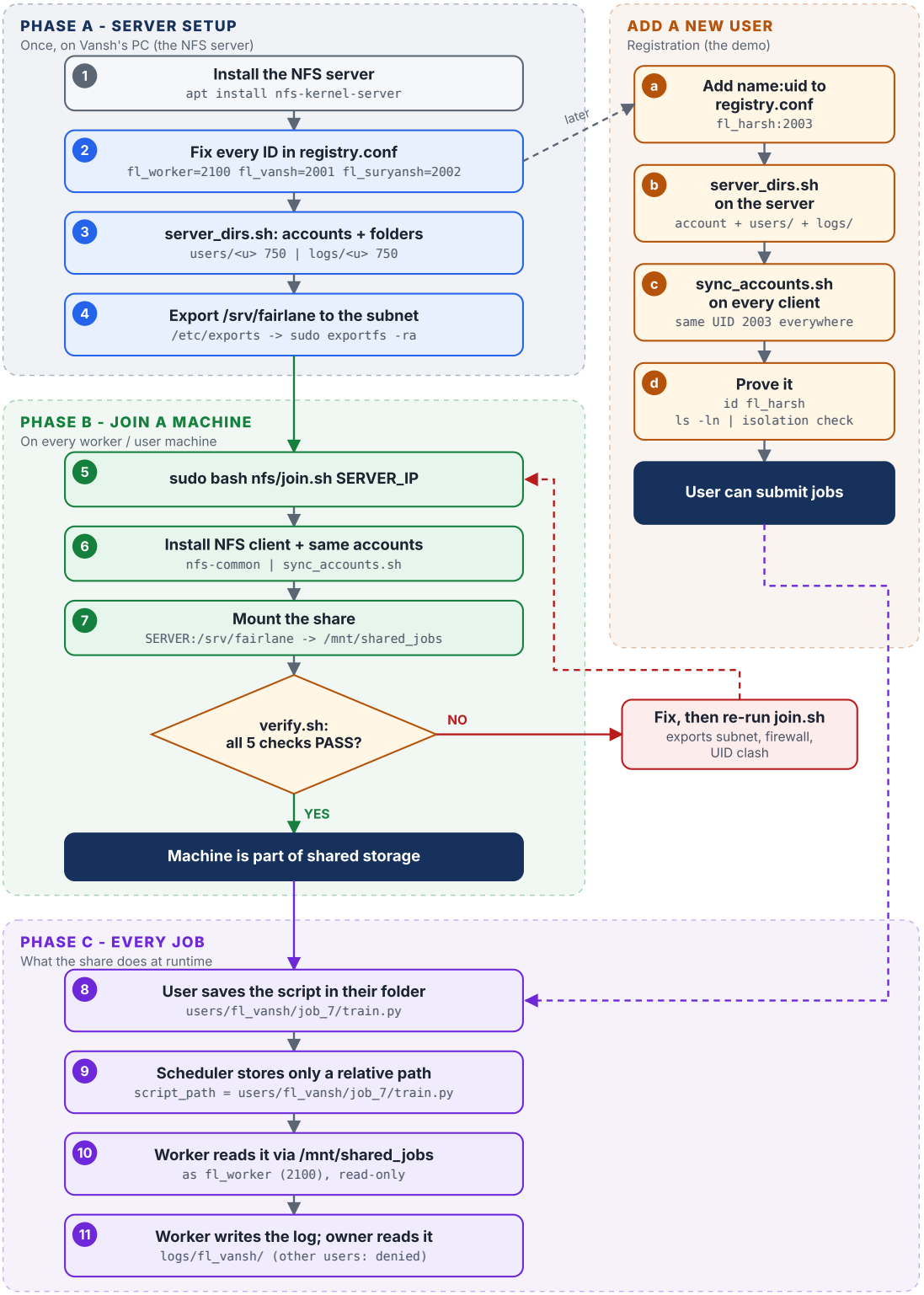
ONE-LINE SUMMARY FOR THE EVALUATOR

Registration is: assign a number in the registry, run one script on the server and one on each client. Because NFS identifies people by number, using the same number everywhere is what keeps every user's files private and the worker able to read them.

9 Summary flowchart

The whole NFS track on one page: set up the server once (A), join each machine (B), and then every job uses the share (C). Registering a new user (right) plugs into the same scripts.

> ***CURRENT STATUS*** *Phases A and B, and the registration loop, are built and tested. Phase C is the target design: in today's live scheduler demo the worker agent still reads demo scripts from a local folder (`~/fairlane_jobs`) and keeps job logs in `/tmp`. Pointing the agent at `/mnt/shared_jobs` and running it as `fl_worker` is the next integration step.*



10 Likely evaluator questions

Why NFS and not just copying files to the worker?

The scheduler only passes a path, and the worker is chosen at the last moment. With NFS every worker already sees every file, so there is no transfer code and no stale copies.

Why do UIDs have to match?

NFS sends numbers, not names. If the numbers differ, files appear to belong to someone else and permissions break without any error. registry.conf prevents that.

Can the worker modify a student's code?

No. users/<u> is 750 with group compute_workers, so the worker can only read. It writes only into logs/<u>.

What stops me using sudo on my laptop to read everything?

root_squash: the server treats a client's root as nobody. The remaining gap (faking a UID with root) needs Kerberos, which we list as a known limitation.

What if a number is already taken on some machine?

sync_accounts.sh refuses and prints an error. We pick the next free number in the registry and use it everywhere.

What happens if the NFS server goes down?

Machines keep the mount, but anything touching the share waits until the server is back (that's how a normal NFS mount behaves). The server is a single point of failure, which is acceptable for a lab; a production setup would use replicated or highly available storage.